GB20602 - Programming Challenges

Week 0 - Course Details

Claus Aranha
caranha@cs.tsukuba.ac.jp

College of Information Sciences

(last updated: April 17, 2025)

Version 2025

Course Introduction

What is the goal of this course?

- Course Objective: Improve your skill by writing many programs;
- How? Write a program that solves a puzzle (programming challenge)
  1. Read and understand the puzzle;
  2. Think what program solves the puzzle;
  3. Write the program;
  4. Test the result of the program;
- Why?
  1. We use in practice the algorithms learned in the second year.
  2. We learn how the algorithm works in practice.
  3. We learn how to debug from data.

Automated Judge (AJ)

This course uses an Automated Judge (AJ) to check your homework.

- An AJ is a program that checks if code is correct:
  - Ex: AtCoder, Aizu Online Judge, Topcoder, Codeforces, etc.
- The AJ checks your program agains Secret Test Data.
- The AJ tells you if your code is correct or incorrect.

When your program is incorrect: Be Careful!

- Debug your program carefully;
- Create your own test data;
- Think before submitting again;

Class Requirements

- You have basic programming knowledge (C++, Java or Python);
  - This is a hard programming class
  - Data structures and algorithms are important
- You will do homework every week;
  - Minimum 2 programs per week;
  - Debug carefully;
- You will not copy the homework;
  - Copy homework from other students will be penalized;
  - Copy homework from the internet will be penalized;
  - Penalties include failing the course, and failing the semester;
- No final exam – only homework

Hint: Do your homework early!

Homework: What are Programming Challenges?

It is a puzzle that you solve by writing a program.

The program reads the input, and must write the correct output.

- Difficulty 1: The output must be exactly correct.
- Difficulty 2: The test input is not known.
- Difficulty 3: There is a maximum execution time.

In general, a correct program will be small (< 200 lines).

Programming Challenge: Example

Problem Description

Given a list of n integers $(x_1, x_2, x_3, \ldots, x_n)$, you have to calculate the sum from x_a to x_b, many times.

Input Description

- First you receive 2 integers in a line: n, m. $(1 \leq n, m \leq 10000)$
- Next you receive n integers in a line: x. $(-10000 \leq x \leq 10000)$
- Next you receive m lines with two integers: a, b. $(1 \leq a, b \leq n)$

Output Description

For each query, you must print the sum of numbers between x_a and x_b in a single line.

Programming Challenge: Input/Output Example

Input

```
10 2
3 5 2 -1 10 -5 3 10 2 9
1 4
3 8
```

Output

```
9       # 3 + 5 + 2 + -1
19      # 2 + -1 + 10 + -5 + 3 + 10
```

Programming Challenge: Simple solution

```
int main() {
  int[10000] numbers;
  int n, m, a, b;
  int sum;

  cin >> n >> m;
  for (int i = 0; i < n; i++) { cin << numbers[i]; } # read input
  for (int i = 0; i < m; i++) {                      # read queries
    cin << a << b;
    sum = 0;
    for (int j = a - 1; j < b; j++) {
      sum += numbers[a-1];
    }
    cout >> sum >> endl;                             # output
  }
}
```

Programming Challenge: Simple solutions, simple problems

- Problem 1: What if $a > b$?

  The problem description says "between a and b" but does not say which one is bigger. If $a > b$, the simple program will give a wrong answer!

  Solution: Change the program: if $a > b$, then $a = t, a = b, b = t$.

- Problem 2: What if m is big?

  Each query costs $|a - b|$. If $m \equiv n$ and $|a - b| \equiv n$, then the program costs $n * n = 10000 * 10000 = 10^8$

  Solution: Use memoization to store the sums, and calculate the difference of the sums!

Programming Challenge: Memoization Example

```
int main() {
  int[10000] numbers; int[10000] memo; int n, m, a, b; int sum;
  cin >> n >> m;

  cin >> numbers[0]; memo[0] = numbers[0];
  for (int i = 1; i < n; i++) {
    cin << numbers[i];
    memo[i] = memo[i-1] + numbers[i]                    # memo table
  }

  for (int i = 0; i < m; i++) {                         # read queries
    cin << a << b;
    if a > b { int t = a; a = b; b = t; }               # sort a and b
    cout << memo[b] - memo[a - 1] << endl;              # sum in O(1)
  }
}
```

Topics in this course

1. Introduction
2. Data Structures
3. Search Problems
4. Dynamic Programming
5. Graphs Problems (Graph Structure)
6. Graph Problems (Graph Search and Flow)
7. String Manipulation
8. Math Problems
9. Geometry Problems
10. Final Remix

About the Lecturer



- Name: Claus Aranha;
- Country: Brazil;
- Research Topics:
  - Evolutionary Computation;
  - Artificial Life;
- Hobbies:
  - Game Programming;
  - Geocaching;
- webpage:
  `http://conclave.cs.tsukuba.ac.jp`

Why programming challenges?

- Competition: Competitive programming started in 1980. Today, many universities in the world compete, including Tsukuba.
- Study: You can use automated Judges to study and improve your programming ability. (AtCoder, Codeforces, etc);
- Recruitment: Many companies use programming challenges during the recruitment process.
- For Fun: It is fun to solve puzzles and to program!

Extra: Join the Tsukuba ICPC Team!

What is ICPC?



If you like these exercises, please consider joining the ICPC contest!

ICPC (International Collegiate Programming Contest) is the largest and most traditional programming competition between universities.

More than 50.000 students from all over the world participate every year.

Contest Website: `https://icpc.iisf.or.jp/`

Extra: Join the Tsukuba ICPC Team!

Program and see the world!



- Requirements: Team of 3 students, any course;
- Schedule:
  - Online Preliminary Contest: July
  - Japanese National Contest: November/December
  - Asian Pacific Championship: March
  - World Final: June (next year)
- Contact me if you're interested!

Course Organization

Outline

1. Course Schedule
2. Course Materials
3. How to submit problems
4. Grading
5. Office Hours and Teacher Communication

What you will do every week

Monday – Get materials from manaba. Check the problems;

Tuesday – 3C205/3C206: Solve the problems, Ask questions to the teacher;

Entire Week – Submit the problems on DMOJ;

Next Tuesday – Homework Deadline

Course Dates and Deadlines

Course Dates

- 4/15, 4/22, 5/07, 5/13, 5/20, 5/27, 6/3, 6/10, 6/17, 6/24;
- No final exam;

Deadlines

- Deadline for homework: Every Tuesday, 23:00

Course Materials on Manaba

- Download PDF file from Manaba
- Video Link on Manaba
- Read Announcements
- Answer Feedback surveys
- Use Forum for questions

Websites to submit homework

DMOJ online judge

- https://judge.conclave.cs.tsukuba.ac.jp/
- Create an account using your university e-mail (sXXXXXXX@u.tsukuba.ac.jp)
- We need to approve accounts by hand, so create your account quickly!
- See Manaba for more information.

About Course Language

Natural Language

- Materials and Homework: English
- Video and manaba: Japanese
- E-mail, feedback: English/Japanese;

Programming Language

- Official Languages: C, C++, Java, Python;
- If you want to use other languages, contact me.

Reference Books

Textbook:

- textbook: Steven Halim, Felix Halim,"Competitive Programming", 4th edition. `https://cpbook.net/`

Other books:

- Steven S. Skiena, Miguel A. Revilla,"Programming Challenges", Springer, 2003
- 秋葉拓哉、 岩田陽一、 北川宜稔,『プログラミングコンテストチャレンジブック』
- 渡部有隆、『オンラインチャレンジではじめるC/C++プログラミング入門、Online Programming Challenge!』 (ISBN978-4-8399-5110-8)
- 渡部有隆、『プログラミングコンテスト攻略のためのアルゴリズムとデータ構造』（ISBN978-4-8399-5295-2）

Grading Rules

Base Grade

Your base grade is based on the number of accepted homework programs you submit:

- C grade: 2+ accepted problems every week;
- B grade: 3+ accepted problems every week;
- A grade: 5+ accepted problems every week;
- A+ grade: 7+ accepted problems every week;

Important!!

- Every week means "X problems every week" (not “average”);
- You can submit problems late, with a penalty;

Grading Rules

Late Penalty

You can submit problems late. But there is a penalty.

If the number of total late programs $\geq$ 25% of total programs, your grade will lower 1 step.

You will not fail the course for late programs.

Example:

- Student (1) submitted 45 problems, minimum 5 problems per week. 5 problems are late. $5 \leq 45 * 0.25$, no penalty. Grade A.
- Student (2) submitted 46 problems, minimum 5 problems per week. 16 problems are late. $16 \geq 46 * 0.25$, penalty. Grade B.
- Student (3) submitted 24 problems, minimum 2 problems per week. 10 problems are late. $10 \geq 24 * 0.25$, penalty. Grade C (will not fail).

Grading

Plagiarism

The assignments are individual. You must write your programs by yourself.

You can do this

- Ask for ideas to your friends;
- Ask for ideas in the MANABA forum;
- Ask for help with a bug;

You can NOT do this

- Copy a solution from the internet;
- Copy a solution from your friends;
- Give your code to a friend;

Students who do plagiarism will fail the course, and suffer penalties from the university.

About these Slides

These slides were made by Claus Aranha, 2024. You are welcome to copy, distribute, re-use and modify this material. (CC-BY-4.0)

Individual images in some slides might have been made by other authors. Please see the following pages for details.

Image Credits I